

Lecture 6: Error Detection/Correction

CRC

EC 441 - Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Apply the single-bit error correction theorem to determine minimum codeword length
- Quantitatively compare error detection, correction, and repetition coding
- Compute block error probabilities using the binomial distribution
- Describe CRC codes using polynomial notation: $G(x)$, $T(x)$, $M(x)$, $R(x)$
- Perform CRC encoding and decoding using polynomial (modulo-2) division
- Identify standard CRC polynomials and their error detection capabilities

Quick Review: Block Codes and $d_{\min}$

Block code (n, k) : k data bits encoded into n -bit codewords

Key parameters:

- **Code rate:** $R_c = k/n$
- **Hamming distance:** Number of bit positions where two words differ
- **Minimum distance** $d_{\min}$: Smallest Hamming distance between any codeword pair

The $d_{\min}$ theorem:

$$e_c + e_d \leq d_{\min} - 1, \quad e_c \leq e_d$$

- To **detect** d errors: need $d_{\min} \geq d + 1$
- To **correct** d errors: need $d_{\min} \geq 2d + 1$

Single-Bit Error Correction Theorem

Theorem: Single-bit error correction requires the codeword length n to satisfy:

$$n + 1 \leq 2^{n-k}$$

If n is less than the minimum value satisfying this inequality, the code **cannot** perform single-bit error correction.

Example: For $k = 1000$ data bits, solve $n + 1 \leq 2^{n-1000}$

Testing $n = 1010$: $1011 \leq 2^{10} = 1024$ – satisfied!

⇒ Need $n - k = 10$ check bits minimum for single-bit error correction of 1000-bit frames.

Proof: Sphere-Packing Argument

For single-bit error correction ($e_c = 1$):

Each codeword needs $n + 1$ words reserved for itself:

- The codeword itself (0 errors)
- The n words that differ in exactly one bit position

Since there are 2^k codewords, we need at least $2^k(n + 1)$ distinct words.

But there are only 2^n possible words total. Therefore:

$$2^k(n + 1) \leq 2^n$$

Dividing both sides by 2^k :

$$n + 1 \leq 2^{n-k} \quad \blacksquare$$

Discussion of the Theorem

Important properties:

- This is a **necessary** condition (not sufficient)
 - Satisfying the inequality does not guarantee a code exists
- As n increases, 2^{n-k} grows exponentially $\Rightarrow$ the inequality eventually holds
- Puts a **limit on the maximum frame length** for a given number of check bits

Intuition: The check bits must encode enough information to identify **which** bit (if any) was flipped. With $n - k$ check bits, we can distinguish 2^{n-k} cases. We need to identify $n + 1$ cases (no error, or error in position $1, 2, \dots, n$).

BER Comparison Setup

Compare three error control methods:

- ① Single-bit error correction
- ② Single-bit error detection (with retransmission)
- ③ Rate 1/3 repetition coding (with error correction)

Parameters:

- Frame size: $n = 1000$ bits
- Channel BER: $p = 10^{-6}$
- Use the most efficient codes possible
- Retransmit if there is a detected error

Metrics:

- **Utilization** U : fraction of channel used for data
- **Residual BER** P_e : probability of bit error after decoding

Block Error Probabilities

Errors in each bit are independent $\Rightarrow$ number of errors in a block is **binomial**:

$$f(w) = \binom{N}{w} p^w (1-p)^{N-w}, \quad N = 1000, \quad p = 10^{-6}$$

Key values:

Errors w	$f(w)$
0	0.999
1	0.001
2	$\approx 10^{-6.3}$
3	$\approx 10^{-9.8}$

Most blocks have **zero errors**. Blocks with ≥ 2 errors are extremely rare.

Method 1: Single-Bit Error Correction

From the theorem: $(1000 + 1) \leq 2^{n-k}$, so $n - k = 10$ check bits.

Utilization: No retransmissions needed:

$$U = \frac{k}{n} = \frac{990}{1000} = 0.990$$

Residual BER: Errors occur when ≥ 2 bits are wrong (can only correct 1):

$$P_e = \frac{1}{N} \sum_{q=2}^N q \cdot f(q) \approx \frac{2 \times 10^{-6.3} + 3 \times 10^{-9.8}}{1000} \approx 10^{-9}$$

Method 2: Single-Bit Error Detection

Use $n - k = 1$ check bit (parity bit code).

Utilization: Retransmissions follow geometric distribution with success probability $\approx 1 - 10^{-3}$:

$$U = \frac{k}{n \cdot E[M]} = \frac{999}{1000 \times \frac{1}{0.999}} = \frac{998}{1000} = 0.998$$

Residual BER: Accepted frames may have ≥ 2 undetected errors (even number for parity):

$$P_e \approx 10^{-9}$$

Same order as error correction, but **higher utilization!**

Method 3: Repetition Coding

Rate 1/3 repetition: $n = 999$, $k = 333$. Each bit is transmitted 3 times.

Utilization:

$$U = R_c = \frac{1}{3} = 0.333$$

Residual BER: A bit error occurs when ≥ 2 of 3 copies are flipped:

$$P_e = \binom{3}{2} p^2 (1 - p) + p^3 = 3 \times 10^{-12} + 10^{-18} \approx 3 \times 10^{-12}$$

Better P_e but at enormous throughput cost.

BER Comparison Results

Method	Utilization U	Residual P_e
Single-bit error detection	0.998	10^{-9}
Single-bit error correction	0.990	10^{-9}
Rate 1/3 repetition coding	0.333	10^{-12}

Parameters: $n = 1000$ bits, channel BER = 10^{-6}

BER Comparison Discussion

Key insights:

- The **rate 1/3 code is very inefficient** in throughput
- **Error detection** achieves $\sim 5\times$ higher utilization than correction for the same P_e
- For networking (where retransmission is possible), **error detection wins**

When to use each:

- **Error detection + retransmission:** Networking (Ethernet, WiFi, TCP)
- **Error correction:** Real-time systems (voice, video), deep space
- **Repetition:** Ultra-reliable, low-rate applications (safety critical)

Design Example 3: Problem Statement

An error control code uses the following set of codewords:

$$\{00000, 11100, 11011, 00111\}$$

- a) What is the rate of the code?
- b) What is $d_{\min}$?
- c) If used for detection only, how many bit errors are guaranteed to be detected?
- d) If used for correction only, how many bit errors are guaranteed to be corrected?
- e) What should you do if you receive 01010 and are using error correction?

Design Example 3: Answers

(a) $R_c = k/n = 2/5 = 0.4$ (4 codewords $= 2^2$, so $k = 2$, $n = 5$)

(b) All pairwise distances: 3, 4, 3, 3, 4, 3 $\Rightarrow d_{\min} = 3$

(c) Detection only: $e_d = d_{\min} - 1 = 2$ errors

(d) Correction only: $e_c = \lfloor (d_{\min} - 1)/2 \rfloor = 1$ error

(e) 01010 is distance 2 from both 00000 and 11011

$\Rightarrow$ **Cannot reliably correct** (equidistant from two codewords)

Options: request retransmission or guess (50% chance)

Check: $n + 1 = 6 \leq 2^{n-k} = 2^3 = 8 \checkmark$ (necessary condition met)

CRC Introduction

Cyclic Redundancy Check (CRC): Most widely used error detection codes in networking.

Treat bit strings as **polynomials over GF(2)** (binary arithmetic, no carries).

Polynomial Notation:

Symbol	Meaning	Size
$M(x)$	Message (data) polynomial	k bits
$G(x)$	Generator polynomial	$r + 1$ bits
$R(x)$	Remainder (CRC checksum)	r bits
$T(x)$	Transmitted frame (codeword)	$n = k + r$ bits

Key principle: Valid codewords $T(x)$ are divisible by $G(x)$:

$$T(x) \equiv 0 \pmod{G(x)}$$

Binary Polynomial Representation

Bit strings are represented as polynomials where each bit is a coefficient:

$$10011 \longleftrightarrow x^4 + x + 1$$

$$1101 \longleftrightarrow x^3 + x^2 + 1$$

$$101000 \longleftrightarrow x^5 + x^3$$

Conventions:

- Leftmost bit = highest-degree term
- Each bit position i corresponds to coefficient of x^i
- A k -bit message has polynomial of degree $\leq k - 1$

GF(2) Arithmetic

All arithmetic is modulo-2 (XOR):

- Addition: $1 + 1 = 0$, $1 + 0 = 1$, $0 + 0 = 0$
- **No carries or borrows**
- Addition and subtraction are the **same operation** (XOR)

Example – Addition:

$$(x^3 + x^2 + 1) + (x^3 + x + 1) = x^2 + x$$

$$1101 \oplus 1011 = 0110$$

Example – Multiplication:

$$(x^2 + 1)(x + 1) = x^3 + x^2 + x + 1$$

$$101 \times 11 = 1111$$

CRC Encoding Steps

Given message $M(x)$ and generator $G(x)$ of degree r :

Step 1: Multiply $M(x)$ by x^r (append r zero bits to message)

Step 2: Divide $x^r M(x)$ by $G(x)$ to get remainder $R(x)$:

$$x^r M(x) = Q(x) \cdot G(x) + R(x)$$

Step 3: Transmitted frame:

$$T(x) = x^r M(x) + R(x)$$

Why this works:

$$T(x) = Q(x) \cdot G(x) + R(x) + R(x) = Q(x) \cdot G(x)$$

So $T(x)$ is divisible by $G(x)$ ✓ ($R(x) + R(x) = 0$ in $\text{GF}(2)$)

CRC Encoding Example

$$M = 1101 (x^3 + x^2 + 1), \quad G = 1011 (x^3 + x + 1), \quad r = 3$$

Step 1: Append 3 zeros: $x^3 M(x) = 1101\mathbf{000}$

Step 2: Modulo-2 long division of $1101000 \div 1011$:

$$\begin{array}{r} 1011 \) \ 1101000 \\ \underline{1011} \leftarrow \text{XOR} \\ 1100 \\ \underline{1011} \\ 1110 \\ \underline{1011} \\ 1010 \\ \underline{1011} \end{array}$$

At the receiver: Divide received frame $\hat{T}(x)$ by $G(x)$

- If remainder = 0: **Accept** (no detected error)
- If remainder $\neq 0$: **Reject** (error detected $\Rightarrow$ request retransmission)

Verification of example: Divide 1101001 by 1011:

Remainder = 000 ✓ (valid codeword)

If errors occurred: e.g., receive 1001001 (bit 5 flipped)

Divide by 1011 $\Rightarrow$ remainder $\neq 0 \Rightarrow$ error detected!

Why CRC Works – Error Polynomial

If $\hat{T}(x) = T(x) + E(x)$ where $E(x)$ is the error pattern:

$$\hat{T}(x) \bmod G(x) = \underbrace{T(x) \bmod G(x)}_{=0} + E(x) \bmod G(x)$$

An error is **undetected** only if $E(x)$ is divisible by $G(x)$:

$$\text{Undetected error} \iff G(x) \mid E(x)$$

A well-chosen $G(x)$ makes this extremely unlikely!

Design principle: Choose $G(x)$ so that common error patterns (single bits, bursts, odd numbers of errors) are **never** divisible by $G(x)$.

CRC Error Detection Capabilities

For a generator $G(x)$ of degree r :

Error Type	Detection Condition
All single-bit errors	$G(x)$ has more than one term
All double-bit errors	$G(x)$ does not divide $x^k + 1$ for $k \leq n$
All odd numbers of errors	$G(x)$ has $(x + 1)$ as a factor
All burst errors $\leq r$ bits	Always (for any $G(x)$ of degree r)
Burst errors of length $r + 1$	Probability $1 - 2^{-(r-1)}$
Burst errors of length $> r + 1$	Probability $1 - 2^{-r}$

Burst error: Contiguous sequence of l bits where the first and last bits are in error (intermediate bits may or may not be).

Standard CRC Polynomials

Name	Polynomial	r	Common Use
CRC-8	$x^8 + x^2 + x + 1$	8	ATM header
CRC-16	$x^{16} + x^{15} + x^2 + 1$	16	USB, Modbus
CRC-CCITT	$x^{16} + x^{12} + x^5 + 1$	16	HDLC, Bluetooth
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + \dots$	32	Ethernet, WiFi

CRC-32 (full):

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

CRC-32 in Practice

Used in Ethernet and WiFi – the most common link-layer protocols.

CRC-32 detects:

- All single, double, and triple bit errors
- All odd numbers of errors
- All burst errors of length ≤ 32
- For random errors: undetected error probability $\approx 2^{-32} \approx 2.3 \times 10^{-10}$

Implementation:

- Hardware: **shift registers with XOR gates** (computed at wire speed)
- Software: lookup tables (256-entry table, one byte at a time)
- Adds only 32 bits (= 4 bytes) of overhead per frame

Ethernet Frame Check Sequence (FCS): last 4 bytes of every frame are CRC-32

Key Takeaways (1/2)

1 Single-Bit Error Correction Theorem

- Requires $n + 1 \leq 2^{n-k}$ (sphere-packing bound)
- Necessary condition, not sufficient

2 BER Comparison

- Error detection: $U = 0.998$, $P_e = 10^{-9}$
- Error correction: $U = 0.990$, $P_e = 10^{-9}$
- Repetition: $U = 0.333$, $P_e = 10^{-12}$
- **Detection wins** for networking (retransmission available)

3 CRC Fundamentals

- Polynomial representation over GF(2)
- $T(x) = x^r M(x) + R(x)$, where $R(x) = x^r M(x) \bmod G(x)$

Key Takeaways (2/2)

4 CRC Encoding and Decoding

- Encode: append r zeros, divide by $G(x)$, attach remainder
- Decode: divide received frame by $G(x)$; remainder = 0 means valid

5 CRC Error Detection

- Detects all 1, 2, 3-bit errors (with proper $G(x)$)
- Detects all odd numbers of errors if $(x + 1) \mid G(x)$
- Detects all burst errors of length $\leq r$

6 Standard CRCs

- CRC-16: USB, Modbus ($r = 16$)
- CRC-32: Ethernet, WiFi ($r = 32$, undetected error $\approx 2^{-32}$)
- Implemented in hardware at wire speed

Data Link Layer Protocols:

- Framing and frame formats
- Medium access control (MAC) protocols
- Ethernet (802.3) protocol in detail
- WiFi (802.11) protocol in detail
- CSMA/CD and CSMA/CA
- Bridging and switching

Connection: CRC-32 is the error detection mechanism used in **every** Ethernet and WiFi frame – the Frame Check Sequence (FCS).